Universidad Tecnológica de Bolívar
Ingeniería de Software

Motor de historias interactivas

Parcial práctico: *La última transmisión*

Juan Manuel Rudas Tijera (T00067552)
Docente: Aisner Marrugo
Repositorio: `https://github.com/JuanRudas02/trabajo-ingenieria-software`
Cartagena, 14 de octubre de 2026

---

1. Fundamentos

1.1. Problema, actores y alcance

Un colectivo de escritores quiere publicar una historia interactiva para un festival universitario sin depender de un programador cada vez que cambie una escena. Necesitan escribir la historia en un archivo, que las opciones dependan de lo que el lector ya descubrió y que los errores del archivo aparezcan antes de que alguien juegue.

Los actores son la escritora, que crea y corrige los archivos; el lector, que juega; el diseñador de la experiencia, que define condiciones y consecuencias; y la coordinadora del festival, que no acepta cierres inesperados del programa. La Figura A.1 muestra el contexto.

El alcance de la primera versión incluye cargar historias en JSON, validarlas antes de jugar, mostrar escenas y opciones condicionadas, aplicar efectos booleanos, detectar finales y bloqueos, y reiniciar. Quedan por fuera el retroceso (ver sección 4.4), las cuentas de usuario, la base de datos, el guardado entre ejecuciones, la red, el editor visual, los elementos multimedia y cualquier lenguaje de expresiones.

1.2. Riesgos

R1. *La solicitud de cambio de la hora 6 obliga a rehacer partes del diseño.* Para mitigarlo el sistema se separó en capas, la versión `v1.0` se etiqueta en Git antes de recibir el cambio y se mantiene una suite de regresión que se corre después de cada modificación.

R2. *Un archivo de historia con errores llega al lector.* Para mitigarlo, el archivo se valida completo antes de crear la partida (nueve categorías de error) y el README trae el formato y una historia de ejemplo para copiar.

1.3. Atributos de calidad

El orden de prioridad es el siguiente: primero la confiabilidad, porque la coordinadora exige que el programa no se cierre de forma inesperada; después la modificabilidad del contenido, porque la escritora no programa; luego la claridad de los errores, sin la cual la modificabilidad no sirve; después la facilidad de uso de la consola y, al final, el rendimiento, que no es un problema con historias de pocas decenas de escenas.

1.4. Responsabilidad profesional

El motor nunca escribe sobre los archivos de historia: el texto es del colectivo y el programa solo lo lee. No se guardan datos de los lectores. En cuanto a dependencias, el producto usa solo la biblioteca estándar de Python, y `pytest` (licencia MIT) se usa únicamente para las pruebas, así que no se distribuyen bibliotecas de terceros con el juego. El uso de herramientas de IA se declara en la sección 9.

2. Proceso

Trabajé con un proceso incremental. Hubo tres incrementos internos: `v0.1` con el contenido y la validación, `v0.2` con las reglas de la sesión y `v1.0` con la consola y la integración. Después se hace `v1.1` con la solicitud de cambio.

Lo comparé con cascada. Cascada supone requisitos estables desde el principio, pero la guía anuncia un cambio a mitad de la ventana. Con un incremento funcional y probado en la hora 6, el cambio se aplica sobre algo que ya funciona; en cascada me encontraría con la implementación a medias.

Actividad	Cómo la realicé en esta entrega
Especificación	RF atómicos con criterio de aceptación (sección 4), revisados con una lista de chequeo (atomicidad, verificabilidad y cobertura de N01 a N06) antes de programar.
Desarrollo	Por incrementos. Cada criterio de aceptación se escribió primero como prueba en `pytest`.
Validación	Suite automatizada de 69 casos, más el recorrido manual de los tres finales y de la segunda historia.
Evolución	Etiqueta `v1.0`, análisis de impacto de la solicitud y `v1.1` con la trazabilidad actualizada (sección 8).

3. Agilidad

Usé un tablero Kanban con las columnas Por hacer, En curso, Revisión y Hecho. El tablero está en GitHub Projects, en el mismo repositorio; las capturas inicial y final están en el Anexo C.

ID	Prioridad	Elemento del backlog	Responsable	Estado
B1	Alta	Modelo de dominio y cargador JSON (RF01)	J. Rudas	Hecho
B2	Alta	Validador con nueve categorías de error (RF11)	J. Rudas	Hecho
B3	Alta	Sesión: condiciones, efectos y rechazo sin cambios (RF02–RF08)	J. Rudas	Hecho
B4	Alta	Finales, reinicio y bloqueo (RF09, RF10, RF12)	J. Rudas	Hecho
B5	Media	Consola y entradas inválidas (RF03, RF07)	J. Rudas	Hecho
B6	Media	Segunda historia y pruebas de RNF	J. Rudas	Hecho
B7	Media	Requisitos, modelos, arquitectura e informe	J. Rudas	Hecho
B8	Alta	Solicitud de cambio de la hora 6	J. Rudas	[COMPLETAR: estado]

3.1. Prácticas

Apliqué dos prácticas. La primera fue limitar el trabajo en curso a una sola tarea a la vez en el tablero, para no tener varias piezas a medias cuando llegara el cambio. La segunda fue escribir las pruebas de los criterios de aceptación antes del código: CA01 a CA03 y U1 a U5 se convirtieron en pruebas que fallaban hasta que la funcionalidad estuvo lista. Descarté Scrum con sprints y ceremonias formales, porque en una ventana de 12 horas la planificación, la revisión y la retrospectiva de cada sprint cuestan más de lo que aportan.

3.2. Definición de terminado

Una tarea está terminada cuando su código está en la rama principal, todas las pruebas pasan, los requisitos y la matriz de trazabilidad están actualizados, pasó la lista de chequeo de revisión y el README sigue siendo correcto.

3.3. Aprendizaje

La primera versión de la prueba de robustez, con 1000 entradas aleatorias, falló porque no simulaba el fin de la entrada estándar y se quedaba esperando una respuesta. Como acción de mejora, todas las pruebas de consola usan ahora una fuente de entrada que termina con `EOFError`, que es el mismo caso que el programa ya trata como una salida controlada.

4. Requerimientos

4.1. Requisitos funcionales

ID	Nec.	El sistema debe	Criterio de aceptación
RF01	N01	Cargar una historia desde la ruta de archivo que se indica al ejecutar el programa.	Al ejecutar `jugar.py` con `historias/el_faro.json` se muestran el título y la escena F1.
RF02	N01	Iniciar cada partida en la escena `inicio` con los valores de `variables` del archivo.	Al iniciar, la escena es E1 y las dos variables valen `false`.
RF03	N02	Mostrar el texto de la escena actual.	Después de elegir “Revisar señal” se muestra el texto de E2.
RF04	N02	Mostrar solo las opciones cuyas condiciones son todas iguales al valor actual de cada variable.	CA01: después de “revisar” e “investigar”, en E3 se muestran exactamente “Activar con código” y “Cortar comunicación”.
RF05	N03	Aplicar a las variables los efectos de la opción elegida.	Después de “investigar”, `conoce_origen` vale `true`.
RF06	N03	Llevar a la escena destino de la opción elegida.	Después de “investigar” la escena actual es E3.
RF07	N03	Rechazar una opción inexistente, no habilitada o una entrada no numérica sin cambiar la escena ni las variables.	CA02: en E3 sin conocer el origen, `elegir("activar_codigo")` devuelve `aceptada = false` y el estado queda igual.
RF08	N04	Conservar el valor de las variables durante toda la partida.	El efecto aplicado en E2 determina las opciones de E3 en la misma partida.
RF09	N05	Informar el desenlace al llegar a una escena con `final = true`.	Al llegar a E4, E5 o E6 se muestra “FIN DE LA HISTORIA”.
RF10	N05	Reiniciar restaurando la escena y las variables iniciales, sin datos de la partida anterior.	Después de terminar en E4 y reiniciar, en E3 sin investigar aparece “Activar sin verificar”.
RF11	N06	Validar el archivo antes de crear la partida y, si tiene errores, rechazarlo listando cada uno con su categoría, causa y campo o ID.	CA03: con un destino E99 no se abre la partida y se muestra `[DESTINO_INEXISTENTE] escena E3.opcion cortar.destino`.
RF12	N06	Informar un bloqueo cuando una escena no final no tiene opciones habilitadas, y ofrecer reiniciar.	Con una historia cuya E3 solo tiene “activar_codigo”, al llegar sin investigar se muestra “[BLOQUEO]” y se ofrece reiniciar.

4.2. Requisitos no funcionales

ID y atributo	Requisito	Procedimiento, evidencia y resultado
RNF01 Modificabilidad	Una historia nueva que cumpla el formato se juega hasta un final sin modificar ningún archivo de `motor/`, y el código del motor no contiene IDs ni variables de ninguna historia.	`test_rnf.py::test_RNF01` calcula el SHA-256 de `motor/*.py` antes y después de jugar `el_faro.json` y busca IDs en el código. Además, `git diff -stat - motor/` en el commit que agrega la historia. Resultado: 0 archivos modificados.
RNF02 Claridad de errores	El 100 % de los errores de contenido (18 variantes de las ocho categorías que se detectan al cargar) produce un mensaje con categoría, causa y campo o ID, sin trazas de pila.	`test_rnf.py::test_RNF02`, prueba parametrizada con 18 casos. Resultado: 18 de 18.
RNF03 Confiabilidad	Ninguna terminación no controlada con 1000 entradas aleatorias en la consola (semilla 2026) y ningún cambio de estado con 1000 elecciones inválidas.	`test_rnf.py::test_RNF03`, dos pruebas. Resultado: 0 terminaciones y 0 cambios. Registro en `resultados/ultima_ejecucion.txt`.

4.3. Preguntas y supuestos

P1. ¿Son obligatorios `condiciones` y `efectos` aunque estén vacíos? No obtuve respuesta durante la ventana. Supuesto: sí, porque el contrato los lista como campos de la opción; un mapa vacío se escribe `{}`. Quedó documentado en el README.

P2. ¿Qué se hace con campos que el contrato no menciona, por ejemplo notas de la escritora? Supuesto: se ignoran, porque no afectan las reglas y le facilitan el trabajo editorial.

P3. ¿Puede repetirse el ID de una opción en escenas distintas? La guía responde que sí: la unicidad es dentro de la escena. Lo verifiqué con una prueba.

4.4. Conflicto sobre el retroceso

La escritora quiere poder retroceder siempre y el diseñador quiere decisiones irreversibles. La pregunta de aclaración para el cliente es: “Si se permite retroceder, ¿también deben deshacerse los efectos sobre las variables? ¿Hay escenas que el diseñador quiera marcar como irreversibles?”. Mientras no haya respuesta, el acuerdo provisional es que la primera versión no tiene retroceso: las decisiones son irreversibles y la única forma de volver es reiniciar. No dejé nada implementado a la espera de esa decisión.

4.5. Revisión cruzada

Al revisar los requisitos con la lista de chequeo encontré una ambigüedad. La primera redacción de RF04 decía “mostrar las opciones válidas”, lo que no se podía verificar porque “válidas” también podía entenderse como “bien escritas”. La corregí a “cuyas condiciones son todas iguales al valor actual”. Esa corrección dejó ver un error frecuente: si se evalúa `if variables[k]` en lugar de comparar igualdad, “Activar sin verificar” (que exige `false`) nunca aparece. Agregué la prueba `test_U1_condicion_false_se_evalua_por_igualdad`.

También encontré un defecto en el cargador: el módulo `json` de Python acepta claves repetidas en un objeto y se queda con la última sin avisar. Lo corregí con un `object_pairs_hook` que rechaza las claves repetidas, con su prueba `clave_repetida`.

5. Modelado

Los diagramas están en el Anexo A y usan los mismos nombres que el código (`Historia`, `Escena`, `Opcion`, `Sesion`, `condiciones`, `efectos`).

- Contexto (Figura A.1): el motor es una aplicación local y su único sistema externo es el archivo de historia.
- Casos de uso (Figura A.2): CU01 a CU08, relacionados con RF01 a RF12. No hay caso de uso para retroceder.
- Clases (Figura A.3): las clases del dominio son inmutables y `Sesion` es la única con estado que cambia. La regla de igualdad está en `Opcion.habilitada()`.
- Secuencia (Figura A.4): elegir una opción. La consola convierte el número en un ID y la sesión vuelve a validar antes de aplicar los efectos.
- Estados (Figura A.5): Cargando, Rechazada, EnEscena, Bloqueada y Terminada. Las elecciones inválidas son transiciones al mismo estado, sin efectos.

El mapa narrativo (Figura A.7) describe los datos de la historia; no reemplaza los modelos del software.

6. Arquitectura

El sistema tiene tres capas y cada una depende solo de la que está debajo (Figura A.6). La interfaz (`jugar.py`) lee el teclado, imprime y maneja el menú de reinicio. Las reglas (`sesion.py`) guardan el estado y evalúan condiciones y efectos. El contenido (`cargador.py`, `validador.py`, `modelo.py` y `errores.py`) lee, valida y representa la historia. Las historias son datos y están en la carpeta `historias/`.

El contenido se separó del motor para que la escritora pueda cambiar la historia sin tocar código (N01, RNF01). Las reglas se separaron de la interfaz para que la sesión no imprima ni lea del teclado; por eso CA02 y U1 a U5 se prueban sin consola, y una interfaz web podría usar la misma clase. La prueba práctica es `el_faro.json`: cuatro escenas, una condición y un efecto, que se juegan con el mismo motor sin modificarlo.

6.1. Decisión 1: estilo de arquitectura

Contexto. Se espera un cambio de requisitos a mitad de la ventana y hay que cargar una segunda historia sin editar el motor. *Alternativas.* Un script único; MVC; capas de contenido, reglas e interfaz. *Elección.* Capas. *Consecuencias.* Las reglas se prueban sin entrada ni salida y los cambios de formato quedan aislados en el cargador y el validador. A cambio hay más archivos y una indirección más para un programa pequeño. MVC no se justificaba porque no hay varias vistas.

6.2. Decisión 2: cuándo y cómo validar

Contexto. El lector no debe encontrarse con errores (N06) y la escritora necesita saber qué corregir. *Alternativas.* Validar cada escena cuando se visita; validar todo antes de jugar y parar en el primer error; validar todo antes de jugar y reunir todos los errores. *Elección.* La tercera. *Consecuencias.* Ningún archivo inválido llega a abrir una partida (CA03) y la escritora corrige todo en una sola pasada. A cambio, el validador tiene que seguir revisando aunque la estructura esté parcialmente rota, lo que exige más comprobaciones de tipo. Los bloqueos que dependen del camino recorrido solo se detectan durante la partida (RF12), algo que la guía permite.

7. Implementación y pruebas

Usé Python 3.10 o superior (probado en 3.13) con interfaz de consola. Las bibliotecas son las de la biblioteca estándar (`json`, `dataclasses`, `pathlib`, licencia PSF) y `pytest` 9.1 (licencia MIT), solo para las pruebas.

7.1. Decisión de diseño detallado

`Sesion.elegir(id)` primero busca la opción en la escena actual y verifica que esté habilitada. Solo si las dos comprobaciones pasan crea un diccionario nuevo de variables con los efectos y cambia de escena. Así una entrada inválida nunca deja el estado a medias (U3) y la consola no puede saltarse la regla enviando el ID de una opción oculta (CA02). Las variables se entregan como copia para que nadie pueda modificar

el estado desde fuera, y `reiniciar()` crea una copia nueva de las variables iniciales; si se reutilizara el diccionario cargado del archivo, una partida dejaría rastros en la siguiente (N05). La regla de igualdad vive en `Opcion`, que es la clase que tiene la información, y eso reduce el acoplamiento entre `Sesion` y el formato de las condiciones.

7.2. Pruebas

Caso	Entrada y resultado esperado	Prueba y resultado
U1	E3 con `conoce_origen` verdadero o falso: [activar_codigo, cortar] o [activar_sin_verificar, cortar]	`test_sesion::test_U1_*`, 3 casos, pasan
U2	"investigar" en E2: pasa a E3 con `conoce_origen = true`	`test_sesion::test_U2_*` y `test_CA01`, pasan
U3	Seis entradas inválidas (inexistente, de otra escena, no habilitada, en un final, vacía): estado igual	`test_sesion::test_U3_*`, 6 casos, pasan
U4	22 archivos con errores: `ErrorContenido` con categoría y ubicación	`test_validacion::test_U4_*`, 22 casos, pasan
U5	Terminar en E4 y reiniciar: E1 con variables iniciales	`test_sesion::test_U5_*`, 2 casos, pasan
I1	Archivo real hasta E4, E5 y E6; consola con entradas inválidas y reinicio	`test_integracion::test_I1_*`, 4 casos, pasan

Las pruebas de validación cubren las nueve categorías de error: archivo ilegible o con estructura incorrecta, campo obligatorio ausente, inicio inexistente, ID repetido (de escena, o de opción dentro de su escena), destino inexistente, variable no declarada, valor no booleano, cantidad de opciones incompatible con `final`, y bloqueo durante la partida (`test_bloqueo_en_ejecucion_*`).

7.3. Verificación del producto

Los tres finales se alcanzan con las rutas que indica el README, `el_faro.json` se juega hasta sus dos finales y los tres RNF se cumplen. En la última ejecución pasaron las 69 pruebas, sin fallos (`resultados/ultima_ejecucion.txt`). No hay fallos pendientes conocidos; las limitaciones están en el README.

8. Gestión del cambio

Versión de partida: etiqueta `v1.0`, commit [COMPLETAR: hash]. Solicitud recibida: [COMPLETAR: resumen de la solicitud publicada en la hora 6].

Artefacto	Impacto
Requisitos	[COMPLETAR: RF nuevos o modificados y sus criterios de aceptación]
Modelos	[COMPLETAR: figuras actualizadas]
Componentes	[COMPLETAR: archivos modificados]
Pruebas	[COMPLETAR: escenario U6 y casos extendidos]
Backlog	B8 y [COMPLETAR: subtareas]

Versión resultante: `v1.1`, commit [COMPLETAR: hash]. Evidencia: [COMPLETAR: pruebas y resultado].

9. Trazabilidad, autoría y fuentes

La matriz del Anexo B relaciona RF01 a RF12 y RNF01 a RNF03 con sus modelos, los archivos que los implementan y la prueba exacta que los verifica.

El trabajo es individual. [COMPLETAR: confirmar con el docente que el parcial puede presentarse sin pareja y citar aquí su autorización] Todas las tareas del backlog (B1 a B8) y todos los commits del repositorio `https://github.com/JuanRudas02/trabajo-ingenieria-software` son de Juan Manuel Rudas Tijera.

Fuentes y herramientas: documentación oficial de Python y de `pytest`. Usé un asistente de IA (Claude, de Anthropic) para producir un primer borrador del código, las pruebas, los diagramas UML y este informe. Verifiqué los resultados ejecutando las 69 pruebas automatizadas, recorriendo a mano los tres finales y la segunda historia, y revisando cada sección del informe contra la guía del parcial.

Anexos

Diagramas, matriz de trazabilidad y capturas del tablero. Estas páginas no cuentan dentro del límite de ocho páginas del informe.

Anexo A. Diagramas UML

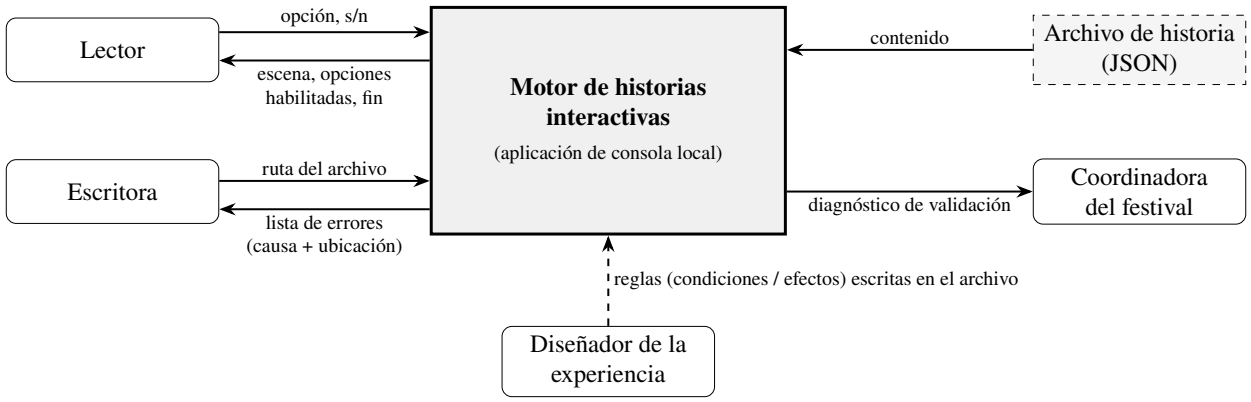


Figura A.1. Diagrama de contexto.

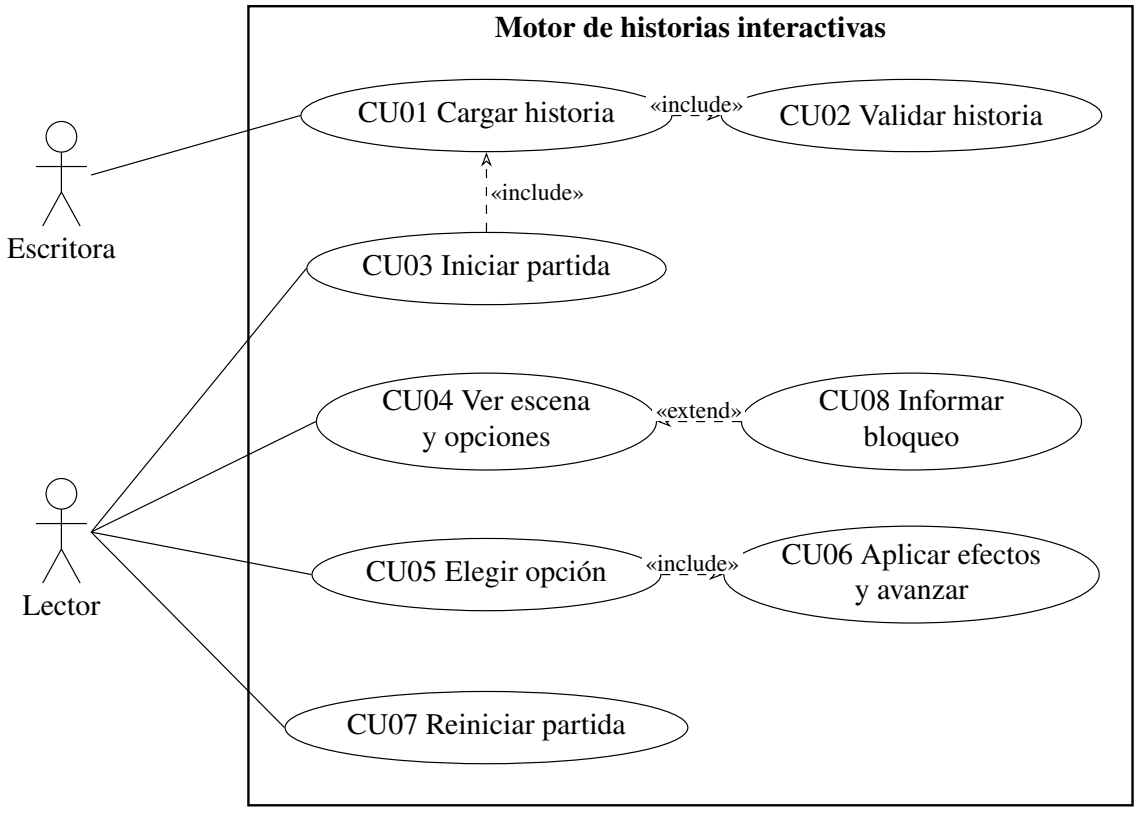


Figura A.2. Diagrama de casos de uso de la primera versión.

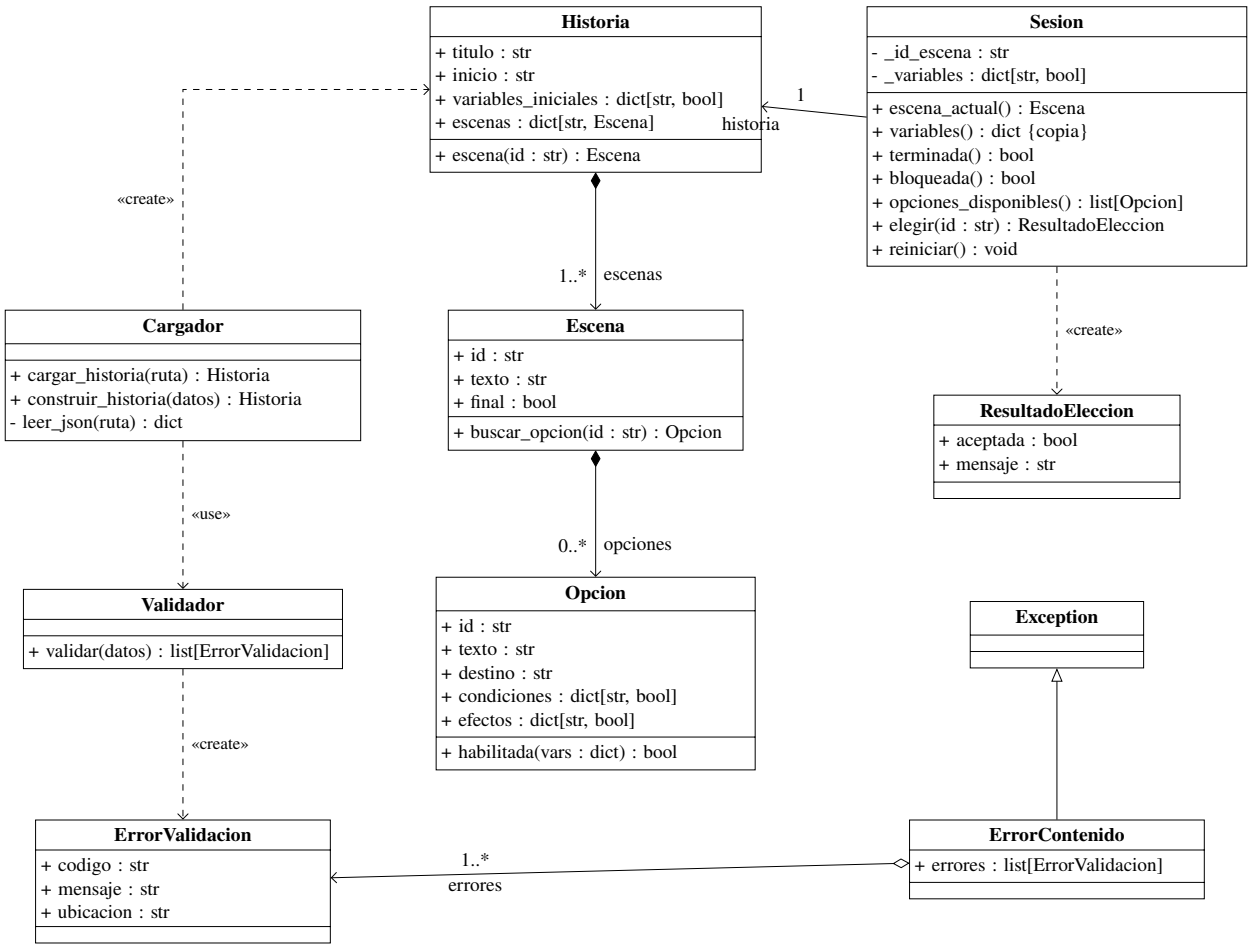


Figura A.3. Diagrama de clases. `cargador.py` y `validador.py` son módulos y se muestran como clases de utilidad.

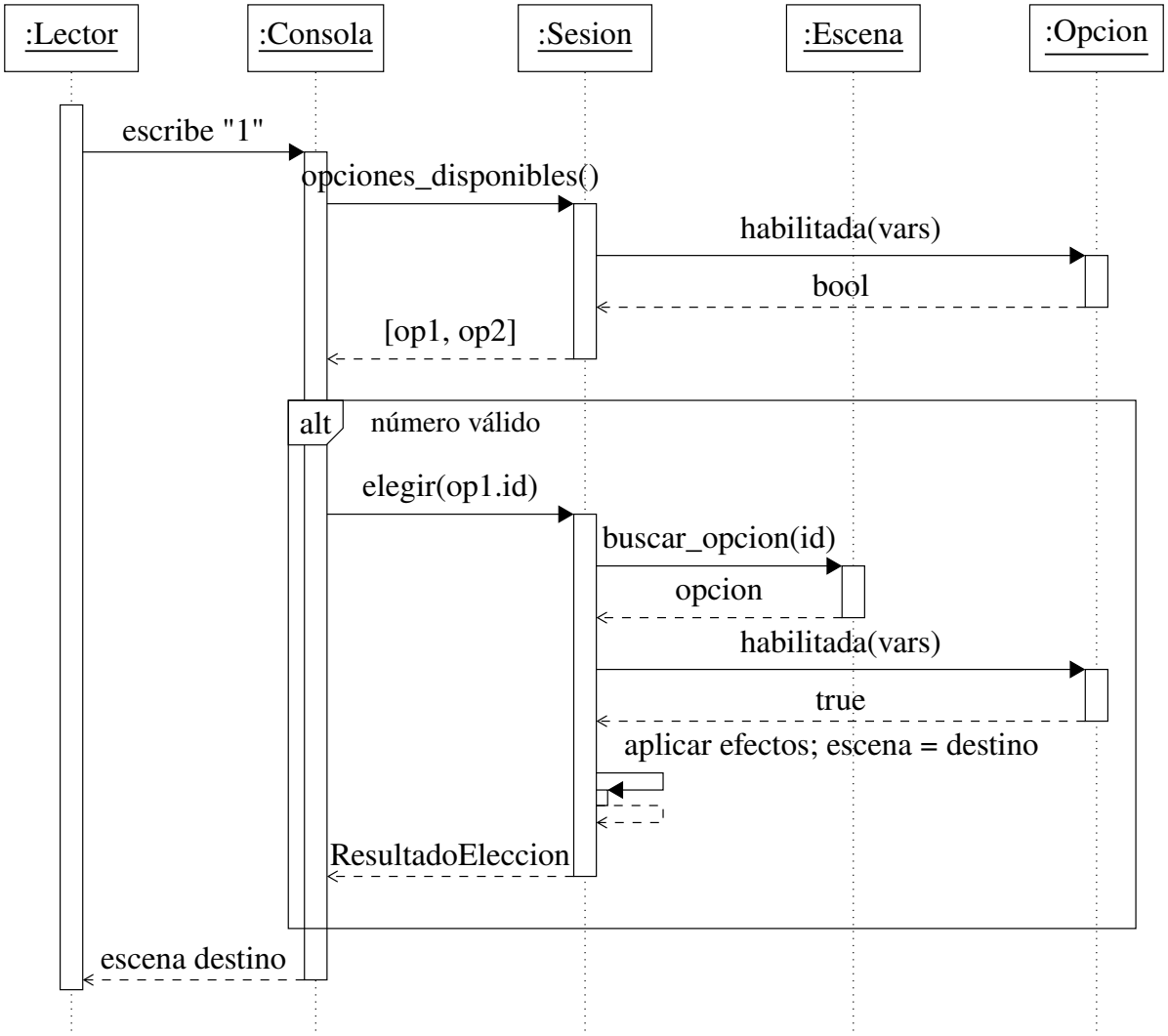


Figura A.4. Diagrama de secuencia: elegir una opción. Si la opción no existe o no está habilitada, `elegir` devuelve `aceptada = false` y no cambia el estado.

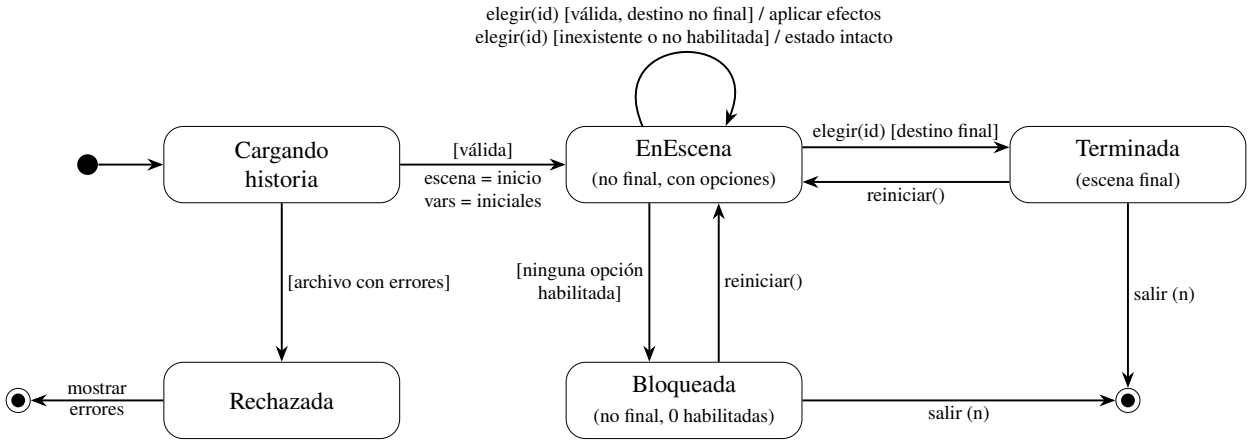


Figura A.5. Diagrama de estados de una partida. “Rechazada” ocurre antes de crear la sesión.

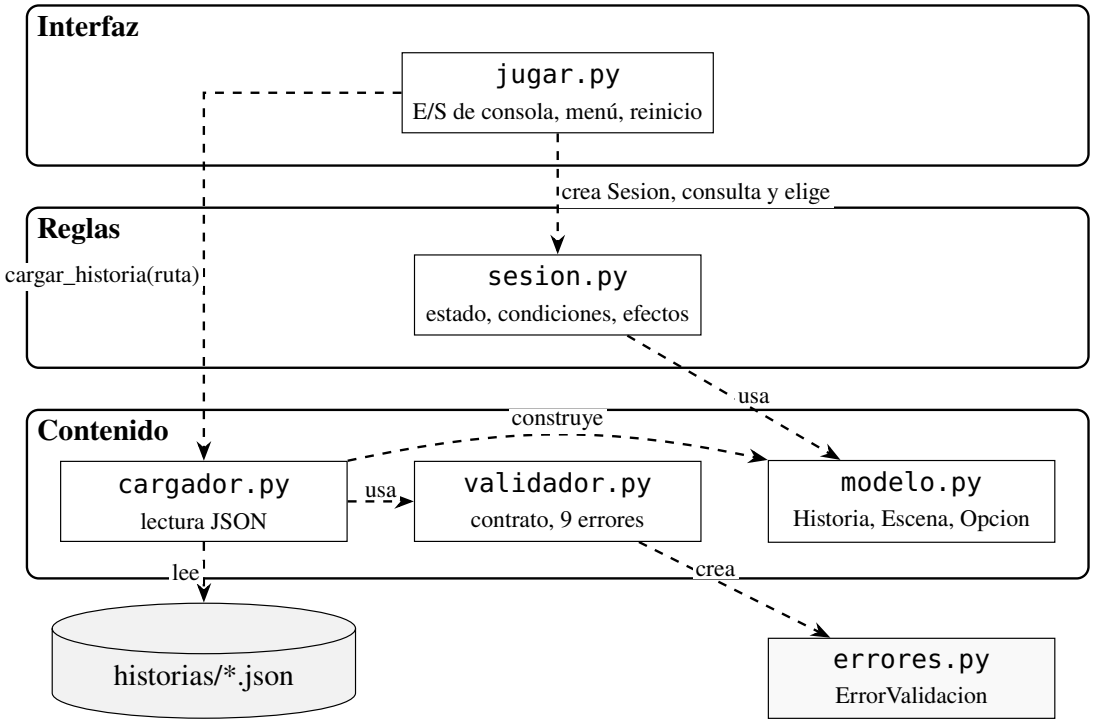


Figura A.6. Diagrama de componentes por capas.

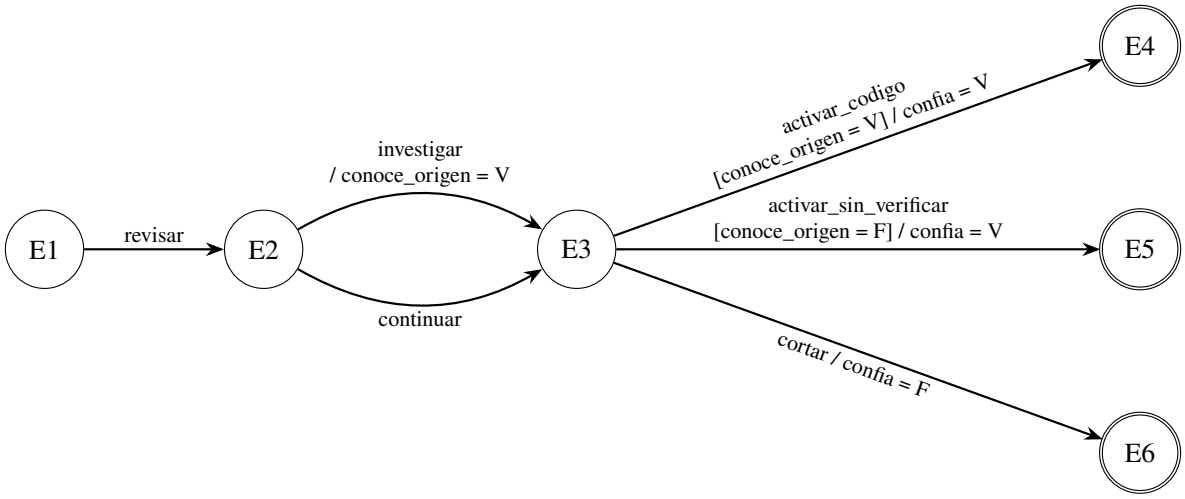


Figura A.7. Mapa narrativo de `la_ultima_transmision.json`. El doble círculo indica una escena final.

Anexo B. Matriz de trazabilidad

Req.	Nec.	Modelos	Componente / archivo	Evidencia de verificación (ruta::prueba)
RF01	N01	CU01, Clases, Comp.	`motor/cargador.py`	`test_integracion.py::test_I1_archivo_real_hasta_cada_final`
RF02	N01	CU03, Estados	`motor/sesion.py` (`reiniciar`)	`test_sesion.py::test_U5_reiniciar_restaura_estado_inicial`
RF03	N02	CU04, Secuencia	`jugar.py`, `Sesion.escena_actual`	`test_integracion.py::test_I1_consola_de_extremo_a_extremo`
RF04	N02	CU04, Secuencia, Clases	`Opcion.habilitada`, `Sesion.opciones_disponibles`	`test_sesion.py::test_U1_*`, `::test_CA01`
RF05	N03	CU06, Secuencia, Estados	`Sesion.elegir`	`test_sesion.py::test_U2_eleccion_valida_aplica_efectos_y_avanza`
RF06	N03	CU06, Secuencia, Estados	`Sesion.elegir`	`test_sesion.py::test_U2_*`, `::test_CA02_*`
RF07	N03	CU05, Estados	`Sesion.elegir`, `jugar.py`	`test_sesion.py::test_U3_*` (6 variantes), `test_rnf.py::test_RNF03_*`
RF08	N04	Clases, Estados	`Sesion._variables`	`test_integracion.py::test_I1_*` (variables finales por ruta)
RF09	N05	CU04, Estados	`Sesion.terminada`, `jugar.py`	`test_integracion.py::test_I1_*` (E4, E5, E6)
RF10	N05	CU07, Estados	`Sesion.reiniciar`	`test_sesion.py::test_U5_*` (2 pruebas)
RF11	N06	CU02, Clases, Comp.	`motor/validador.py`, `errores.py`	`test_validacion.py::test_U4_*` (22 variantes), `::test_CA03_*`
RF12	N06	CU08, Estados	`Sesion.bloqueada`, `jugar.py`	`test_validacion.py::test_bloqueo_en_ejecucion_*`
RNF01	—	Componentes	`motor/*` vs. `historias/el_faro.json`	`test_rnf.py::test_RNF01_*`; `test_integracion.py::test_segunda_historia_*`
RNF02	—	Clases (ErrorValidacion)	`motor/validador.py`	`test_rnf.py::test_RNF02_*` (18 casos)
RNF03	—	Estados	`motor/sesion.py`, `jugar.py`	`test_rnf.py::test_RNF03_*` (2 pruebas, 2000 entradas)
RF-C1	[COMPLETAR: filas de la solicitud de cambio]			

Anexo C. Capturas del tablero

Tablero inicial: [COMPLETAR: tablero_inicial.png]

Tablero final: [COMPLETAR: tablero_final.png]